

具身智能行业和技术发展调研报告 v0.3 Z2b 具 身系统架构

个人学习与研究用途

2026-07-14

装配说明

本入口用于第 18 - 21 章的批次构建，不替代全书 `main.tex`。

目录

第十八章 感知、融合与状态接口	1
18.1 本章任务：把感知结果变成可消费状态	1
18.2 从像素、点和接触到对象状态	1
18.3 时间对齐、融合与统计门控	2
18.4 缺失模态与降级发布	3
18.5 工程失效：数值合法不等于语义正确	3
18.6 知识小结与综述判断	4
第十九章 世界模型、环境表征与可预测状态	5
19.1 先回答：模型预测什么，供谁决策	5
19.2 潜状态空间：从观测重建到动作条件转移	6
19.3 多步 rollout：模型误差如何累积	6
19.4 像素生成、JEPa 与抽象预测的目标差异	7
19.5 怎样证明世界模型真的有用	7
19.6 知识小结与综述判断	7
第二十章 任务规划、结构化中间表示与具身推理	8
20.1 规划的产物不应只是自然语言步骤	8
20.2 四类中间表示及其执行语义	8
20.3 语言相关性必须乘上可执行性	9
20.4 完整例子：取饮料任务的状态与失败分支	10
20.5 多机器人协调不是多份独立计划	11
20.6 知识小结与综述判断	11
第二十一章 技能、控制、安全监督与运行时恢复	12

21.1 运行时不是一条从模型直达电机的箭头	12
21.2 不同时间尺度必须有明确所有权	12
21.3 安全过滤：把建议动作投影回可行集	13
21.4 Simplex 思路：复杂性能控制与简单安全控制分离	13
21.5 异常分类决定恢复，而不是统一“再试一次”	14
21.6 系统案例：抓取策略只是运行时的一环	15
21.7 知识小结与综述判断	15
References	16

第十八章 感知、融合与状态接口

18.1 本章任务：把感知结果变成可消费状态

第 15 章讨论原始模态怎样编码为特征，本章只主讲这些特征怎样成为世界模型、规划器和控制器能够安全消费的状态。检测框、分割掩码、点云簇或触觉图像都是测量解释，尚不是系统状态：它们通常缺少稳定身份、统一坐标系、测量时刻、估计不确定性和有效期。

一个最小状态包可写为

$$\mathcal{S}_t = \{\hat{\mathbf{x}}_t, \mathbf{P}_t, \mathcal{O}_t, \mathcal{C}_t, \mathcal{F}_t, \mathcal{T}_t, v_{\text{calib}}, \mathbf{m}_t\}, \quad (18.1)$$

其中 $\hat{\mathbf{x}}_t$ 与 $\mathbf{P}_t$ 是机器人状态及协方差， $\mathcal{O}_t$ 是带身份的对象集合， $\mathcal{C}_t$ 是接触集合， $\mathcal{F}_t$ 是坐标变换， $\mathcal{T}_t$ 保存测量与发布时间， v_{calib} 是标定版本， $\mathbf{m}_t$ 是逐字段有效性掩码。式 (18.1) 的要点不是字段越多越好，而是每个消费者都能判断“值是什么、在何时何地成立、还能否使用”。

18.2 从像素、点和接触到对象状态

二维检测给出类别与框，实例分割进一步给出像素级对象区域。Mask R-CNN 的并行分类、框回归与掩码分支是典型实例分割结构^[1]，但类别概率不能直接回答对象的六自由度位姿、遮挡程度或可抓取区域。三维点集编码如 PointNet 通过对称聚合保持输入排列不变^[2]；它仍不自动解决传感器外参、稀疏视角和跨帧身份。

对象状态应至少包含

$$o_i = (\text{id}_i, c_i, {}^W\mathbf{T}_{o_i}, \mathbf{v}_i, \Sigma_i, \mathcal{A}_i, t_i, q_i), \quad (18.2)$$

其中 $\mathcal{A}_i$ 表示可供性候选，例如“可抓取区域”或“可倾倒方向”， q_i 表示质量与来源标志。可供性不是类别的固定属性：同一杯子在杯柄被遮挡、内部装有热液体或位于狭窄货架时，可执行动作不同。

触觉又提供另一类局部状态。GelSight 类传感器从弹性体表面形变估计接触几何与力^[3]；它适合判断接触、滑移和局部形状，却不能替代全局视觉定位。把触觉图像当普通 RGB 图像并做强裁剪，可能恰好删掉决定滑移的微小纹理。

表 18.1: 感知输出进入状态包前必须补齐的语义

来源	原始输出	必补字段	主要消费者	不可静默处理的失效
二维视觉	框、掩码、关键点	身份、深度、坐标、遮挡、时间	任务规划、抓取	类别正确但位姿错误
三维感知	点、体素、表面	外参、法向、协方差、可见域	碰撞检查、定位	稀疏区被误当自由空间
触觉/力觉	形变、力矩、滑移分数	接触点、工具坐标、带宽、饱和	柔顺控制、技能终止	饱和值被当真实力
本体状态	关节、速度、电流	单位、顺序、零位、驱动器状态	估计器、控制器	版本错配仍数值合法

18.3 时间对齐、融合与统计门控

多模态融合首先是时序问题。若相机帧的测量时刻为 t_m 、状态包发布时间为 t_p ，系统必须按运动模型把测量传播到共同参考时刻，或明确保留其原始时刻；直接拼接“最新值”会制造不存在的同步快照。第 8 章的滤波与平滑给出了估计基础，本章强调发布接口。

对测量残差 $\boldsymbol{r} = \boldsymbol{z} - h(\hat{\boldsymbol{x}})$ 和创新协方差 $\boldsymbol{S}$ ，常用 Mahalanobis 门控为

$$d^2 = \boldsymbol{r}^\top \boldsymbol{S}^{-1} \boldsymbol{r} \leq \chi_{k,1-\alpha}^2.$$

(18.3)

式中 k 是残差维数， α 是拒绝概率。该门只能说明测量与当前统计模型是否一致，不能证明对象身份或坐标语义正确；系统性外参偏差甚至可能长期“稳定地错误”。Kalman 滤波的协方差更新^[4]和机器人状态估计中的流形、时间与噪声建模^[5]都要求这些假设显式化。

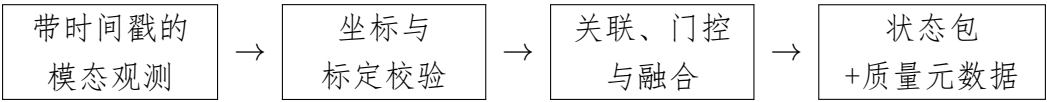


图 18.1: 从测量到可消费状态的生产链。任何校验失败都应产生降级或拒绝事件，而非填入默认零值。

18.4 缺失模态与降级发布

缺失模态至少分为四类：传感器未发布、消息迟到、数值越界、语义质量不足。四者不能统一写成零。以触觉为例，零接触力与“触觉离线”含义相反；以前一帧填充又会把已结束接触延长。状态包应携带 ‘valid ‘、 ‘age ‘、 ‘source ‘ 和 ‘reason ‘，消费者声明自己允许的最大陈旧时间。

Listing 18.1: 状态包发布器的教学伪代码：显式处理迟到、标定与降级

```

1 def build_state_packet(target_time, measurements, calibration):
2     packet = StatePacket(target_time=target_time)
3     for name, msg in measurements.items():
4         if msg is None:
5             packet.invalidate(name, reason="missing")
6             continue
7         if target_time - msg.stamp > MAX_AGE[name]:
8             packet.invalidate(name, reason="stale", age=target_time-msg.
stamp)
9             continue
10        if msg.calibration_version != calibration.version:
11            packet.invalidate(name, reason="calibration_mismatch")
12            continue
13        aligned = transform_and_propagate(msg, calibration, target_time)
14        if not innovation_gate(aligned):
15            packet.invalidate(name, reason="statistical_outlier")
16            continue
17        packet.update(name, aligned.value, aligned.covariance,
18                      source=msg.sensor_id, stamp=msg.stamp)
19    packet.mode = choose_degraded_mode(packet.validity_mask)
20    return packet

```

18.5 工程失效：数值合法不等于语义正确

最危险的接口故障往往不会产生 NaN：毫米与米混用、相机坐标与世界坐标混用、旧外参配新数据、对象 ID 在遮挡后交换，都能产生范围合理的数值。因而测试应同时覆盖单位、坐标变换闭环、时间单调性、标定版本和对象身份连续性。状态包还应记录最近一次重定位、测量拒绝与传感器降级原因，使第 21 章的运行监督器能够追溯。

18.6 知识小结与综述判断

本章把感知系统的输出从“模型预测张量”提升为带契约的状态产品。核心链条是：模态解释产生候选测量，时间与坐标校验建立可比性，关联与融合给出估计和不确定性，状态包再把有效性与来源交给下游。综述判断是：衡量感知模块不能只看离线 mAP 或分割精度，还要看状态在闭环中的陈旧度、校准性、身份稳定性和降级可用性。

第十九章 世界模型、环境表征与可预测状态

19.1 先回答：模型预测什么，供谁决策

“世界模型”常被泛化为任何能生成未来画面的模型。本章采用更严格的系统定义：若模型学习环境在动作作用下的状态转移，并且预测结果实际进入动作选择、风险评估或恢复，它才是决策链中的世界模型。高保真仿真器、控制器内部动力学模型、视频生成器和语义表征模型都可能提供资产，但用途不同。

表 19.1：四类易被混称为世界模型的系统

类型	主要输入/输出	优化目标	决策接入方式	典型盲区
物理仿真器	状态、动作 → 数值轨迹	物理一致与数值稳定	离线训练、测试、MPC	资产与现实偏差
控制模型	局部状态、控制 → 短期状态	局部预测误差	高频优化与稳定控制	长期语义变化
视频生成模型	图像、条件 → 未来帧	感知似然或生成质量	候选可视化、表征学习	动作可控性与物理可行性
决策世界模型	历史、动作 → 潜状态/奖励/终止	决策充分的预测	想象 rollout、价值或风险	模型偏差被策略利用

19.2 潜状态空间：从观测重建到动作条件转移

对观测 $\mathbf{o}_t$ 、动作 $\mathbf{a}_t$ 和潜状态 $\mathbf{z}_t$ ，典型结构分解为

$$\mathbf{z}_t \sim q_\phi(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{a}_{t-1}, \mathbf{o}_t), \quad (19.1)$$

$$\tilde{\mathbf{z}}_{t+1} \sim p_\theta(\mathbf{z}_{t+1} \mid \mathbf{z}_t, \mathbf{a}_t), \quad (19.2)$$

$$\hat{\mathbf{o}}_t \sim p_\theta(\mathbf{o}_t \mid \mathbf{z}_t), \quad (\hat{r}_t, \hat{c}_t) = g_\theta(\mathbf{z}_t). \quad (19.3)$$

q_ϕ 把新观测吸收到后验状态， p_θ 在没有未来观测时执行想象转移， g_θ 预测奖励和继续概率。PlaNet 用潜动力学从像素执行规划^[6]；DreamerV3 则在学习的世界模型中想象未来并训练 actor-critic^[7]。

潜状态的目标不是压缩率最大，而是保留决策所需变量。重建损失偏爱占像素面积大的背景，接触建立、摩擦变化或细小障碍却可能只占少数像素。可增加奖励、终止、接触和约束违反等辅助头，但每个头都会把训练数据中的标注偏差带入状态。

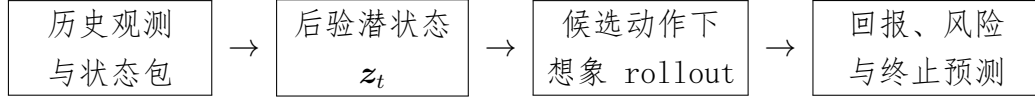


图 19.1：世界模型进入闭环的最小链路。若最后的预测不改变候选动作排序，模型只是辅助表征而非在线决策组件。

19.3 多步 rollout：模型误差如何累积

候选动作序列 $\mathbf{a}_{t:t+H-1}$ 的风险调整目标可写为

$$J = \mathbb{E} \left[\sum_{k=0}^{H-1} \gamma^k \hat{r}_{t+k} \right] - \lambda \sum_{k=1}^H u(\tilde{\mathbf{z}}_{t+k}) - \rho \Pr(\text{constraint violation}), \quad (19.4)$$

其中 $u(\cdot)$ 是模型集成分歧、预测方差或分布外分数。 H 越长并不必然越好：一步小偏差会通过策略选择进入未见区域，随后模型可能给出高回报但不真实的“幻想”。因此工程系统常采用短视窗重规划、真实观测校正和不确定性阈值，而不是一次生成完整长轨迹。

Listing 19.1：带不确定性和硬约束的想象规划教学伪代码

```

1 best = None
2 for action_sequence in propose_candidates(state):
3     latent = world_model.encode(state)
4     score, rejected = 0.0, False
5     for action in action_sequence:
6         latent_ensemble = world_model.predict_ensemble(latent, action)
7         reward = mean_reward(latent_ensemble)
  
```

```
8         uncertainty = ensemble_disagreement(latent_ensemble)
9         if uncertainty > U_MAX or predicts_hard_violation(latent_ensemble):
10             rejected = True
11             break
12         score += reward - RISK_WEIGHT * uncertainty
13         latent = mean_latent(latent_ensemble)
14         if not rejected and (best is None or score > best.score):
15             best = Candidate(action_sequence, score)
16     return best.first_action if best else request_safe_recovery()
```

19.4 像素生成、JEPA 与抽象预测的目标差异

像素生成要求解释纹理、光照和背景，优点是可视化直观，代价是大量容量用于与动作无关的细节。JEPA 类方法在表征空间预测被遮挡区域；I-JEPA 明确采用非生成式目标，由上下文块预测目标块的表征^[8]。这能学习语义表征，但原始 I-JEPA 没有动作条件、奖励或终止变量，不能因“会预测表征”就直接称为机器人动力学世界模型。

抽象预测还可以面向对象关系、接触模式或任务谓词。例如“杯子在托盘上”比每个像素更适合长时程规划；但抽象器若漏掉杯沿裂纹，规划器看不到安全风险。可靠系统往往同时维护多尺度状态：控制层使用连续局部模型，任务层使用对象/谓词状态，视觉模型提供更新证据。

19.5 怎样证明世界模型真的有用

有效证据至少包含三层：一是单步与多步预测在任务相关变量上的校准误差；二是接入规划后相对无模型、真实模型或消融基线的成功率与样本效率；三是分布外、接触突变和传感缺失时的风险与回退。只展示逼真的未来视频不能证明控制收益，只展示平均回报也不能说明失败是否来自模型、规划器或执行器。

19.6 知识小结与综述判断

世界模型把“当前状态”扩展为“候选动作下的可预测状态”。其技术主线是后验编码、动作条件转移、任务相关预测和想象决策。综述判断是：真正的分水岭不在模型是否生成视频，而在预测是否具有动作条件、是否传播不确定性、是否进入闭环，并能在模型不可信时触发真实观测校正或安全恢复。

第二十章 任务规划、结构化中间表示与具身推理

20.1 规划的产物不应只是自然语言步骤

自然语言目标“把冰箱里的饮料拿给客厅的人”省略了对象身份、门状态、抓取约束和失败处理。任务规划的职责，是把意图编译成可由技能库执行、可在状态包上检查、可在失败后恢复的中间表示。第 16 章主讲语义怎样落地到动作接口；本章主讲落地之后的结构、约束和执行反馈，不讨论低层控制细节。

把技能 σ 写成契约

$$\sigma = (\text{Pre}, \text{Param}, \text{Run}, \text{Post}, \text{Fail}, \text{Compensate}), \quad (20.1)$$

其中前置条件决定能否启动，后置条件决定是否成功，失败集合区分可重试、需重规划和必须停机的情形，补偿动作描述已经改变环境后如何回退。没有这些字段，所谓“任务分解”只是一串无法核验的文本。

20.2 四类中间表示及其执行语义

经典自动规划以状态、动作前置条件和效果搜索计划^[9]。HTN 通过方法把复合任务递归分解，适合表达领域程序；行为树通过周期 tick 和 Success/Failure/Running 返回值组织反应式执行^[10]；程序化表示能表达循环、几何计算和 API 组合，但必须面对类型、权限和终止性。

表 20.1: 任务中间表示的能力与代价

表示	强项	执行反馈	易验证内容	主要风险
文本步骤	生成灵活、便于人读	通常非结构化	很少	歧义、漏条件、无法确定调用
HTN/符号	层级分解、前后条件清晰	状态谓词更新	可达性、方法约束	符号状态与现实脱节
行为树	反应式 tick、局部回退	三值或扩展状态	节点契约、局部安全	大树维护与共享状态副作用
受限程序	循环、计算、工具组合	异常、返回值、日志	类型、权限、静态规则	任意代码、超时、API 误用

行为树“可反应”不是因为图画成树，而是控制节点反复 tick 子节点。例如 Sequence 遇到 Failure 立即失败，遇到 Running 保留进度；Fallback 遇到 Success 即返回，失败才尝试下一分支。若叶节点不检查新状态，树仍会机械执行旧计划。

20.3 语言相关性必须乘上可执行性

SayCan 的关键思想是让语言模型对技能与指令的相关性，受到机器人在当前状态下执行该技能的价值/可供性约束^[11]。对候选技能 σ_i 可抽象为

$$i^* = \arg \max_i [\log p_{\text{LM}}(\sigma_i | g, h) + \beta \log p_{\text{exec}}(\text{success} | \mathcal{S}_t, \sigma_i)], \quad (20.2)$$

其中 g 是目标， h 是已执行历史。两项不能互相替代：语言高分不代表机器人够得到，价值高分也不代表动作与任务相关。若安全约束为硬条件，应在排序前直接过滤，而不是仅以小惩罚加入分数。

Code as Policies 进一步让语言模型生成调用感知与控制 API 的程序，可表达反馈循环和几何计算^[12]。这种路线的关键不是“模型会写 Python”，而是暴露给模型的 API 是否受限、参数是否类型化、执行是否在沙箱内、每次调用是否由状态与安全监督器复核。

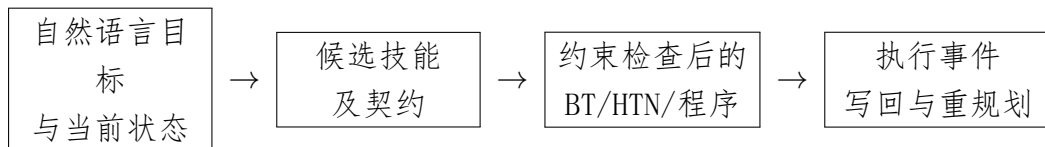


图 20.1: 从语言意图到受约束技能调用的编译链。结构化中间表示位于生成模型与运行时之间。

20.4 完整例子：取饮料任务的状态与失败分支

初始状态为“机器人在厨房外、冰箱门未知、目标饮料身份不确定”。规划器不能直接输出‘grasp(drink)’，而应先定位冰箱、确认门状态、打开门、重新感知、绑定对象身份、验证抓取条件，再执行取放。下面的伪代码刻意把感知写回和补偿分支放在主路径内。

Listing 20.1: 受限任务程序：显式前置条件、写回与失败分支

```

1 def fetch_drink(goal, state):
2     require(goal.destination == "living_room")
3     navigate("fridge", timeout=60)
4     state = observe_state(required=["fridge_door", "drink_candidates"])
5
6     if state.fridge_door == "closed":
7         result = run_skill("open_fridge", force_limit=SAFE_FORCE)
8         if result.code == "handle_not_found":
9             return replan_with_active_perception(result.evidence)
10        if not result.success:
11            return escalate("door_open_failed", result.log)
12
13    state = observe_state(required=["drink_candidates"])
14    target = bind_object(goal.description, state.drink_candidates)
15    if target.confidence < ID_THRESHOLD:
16        return ask_human_to_disambiguate(target.alternatives)
17
18    grasp = plan_grasp(target, constraints=["upright", "collision_free"])
19    result = run_skill("pick", object_id=target.id, grasp=grasp)
20    if result.code == "slip":
21        return run_skill("place_safe_and_regrasp", object_id=target.id)
22    require(observe_state().held_object_id == target.id)
23    return navigate_and_handover(goal.destination, target.id)

```

该程序不是为了展示语法，而是展示五个可审计点：目标被类型化；对象 ID 在感知后绑定；技能调用带约束；失败码决定不同恢复；后置条件由新观测确认。若生成模型只负责候选结构，静态检查器和运行时仍可拒绝越权 API、无界循环或缺失失败分支。

20.5 多机器人协调不是多份独立计划

多个机器人共享对象、通道和工具时，局部可行不等于联合可行。中间表示需增加资源锁、时序约束、通信超时和责任转移。例如一个机器人交付工件后，另一个机器人只有在视觉确认工件进入交接区并取得所有权后才能抓取。语言模型可以生成协作候选，但冲突检测、时间窗和安全互锁应由结构化调度器执行。

20.6 知识小结与综述判断

任务规划的核心产物是可检查的执行结构，而非听起来合理的解释。技能契约定义可执行单元，HTN/行为树/受限程序组织分解与反馈，语言相关性和当前可执行性共同筛选候选，运行事件再写回状态。综述判断是：评价“具身推理”应优先检查计划能否绑定真实对象、通过前后条件、覆盖失败分支并被运行时拒绝，而不是仅评价自然语言推理链是否流畅。

第二十一章 技能、控制、安全监督 与运行时恢复

21.1 运行时不是一条从模型直达电机的箭头

第 20 章产生受约束技能调用，本章讨论调用怎样成为电机命令并在异常时停止。完整运行时至少包含：任务执行器、技能服务器、轨迹/策略、低层控制器、安全过滤器、独立监控器、事件日志和恢复管理器。VLA 或任务规划器只生产候选动作/技能，不拥有绕过速度、力、空间和权限约束的权力。

技能接口应声明输入状态版本、参数单位、允许工作区、最大时长、终止判据、抢占语义和失败码。低层控制器则在更高频率上跟踪目标或调节交互。阻抗控制通过期望的质量 - 阻尼 - 刚度关系管理位姿误差与外力^[13]，操作空间控制在任务坐标中组织动力学^[14]；二者都不能判断“是否抓错人手中的杯子”。

21.2 不同时间尺度必须有明确所有权

表 21.1： 具身运行时的时间尺度、权限与失败响应

层级	典型周期	输出	允许的抢占者	典型失败响应
任务规划	秒至分钟	技能图/程序	人、策略监督器	重规划、请求协助
技能/策略	20 - 500 ms	动作块、目标位姿	安全监督器、超时器	重试、切换技能、撤退
轨迹与控制	0.5 - 10 ms	力矩、速度或位置命令	硬实时安全层	限幅、保持、受控停机
驱动与硬件保护	微秒至毫秒	电流、制动	硬件联锁	断能、制动、故障锁存

高层“停止”必须被翻译为低层可实现的停机轨迹，而不是突然令速度为零并假设机械系统瞬时静止。反之，驱动器故障也必须向上传播为结构化事件，否则任务执行器可能不断重发同一技能。

21.3 安全过滤：把建议动作投影回可行集

令安全集合为 $\mathcal{C} = \{\mathbf{x} : h(\mathbf{x}) \geq 0\}$ 。控制屏障函数方法要求存在 α 使

$$\dot{h}(\mathbf{x}, \mathbf{u}) + \alpha(h(\mathbf{x})) \geq 0. \quad (21.1)$$

对仿射系统 $\dot{\mathbf{x}} = f(\mathbf{x}) + g(\mathbf{x})\mathbf{u}$ ，可把学习策略建议 $\mathbf{u}_{\text{nom}}$ 投影为

$$\mathbf{u}^* = \arg \min_{\mathbf{u}} \frac{1}{2} \|\mathbf{u} - \mathbf{u}_{\text{nom}}\|_{\mathbf{W}}^2 \quad (21.2)$$

$$\text{s. t. } \nabla h^\top(f + g\mathbf{u}) + \alpha(h) \geq 0, \quad \mathbf{u} \in \mathcal{U}. \quad (21.3)$$

该二次规划形式把“尽量接近策略”与“保持安全集合”分开^[15]。但它依赖状态估计、模型和约束可行性：障碍距离过期、制动模型错误或多个约束冲突时，优化器可能无解。无解分支必须定义受控停机或硬件保护，不能回退到未经检查的原动作。

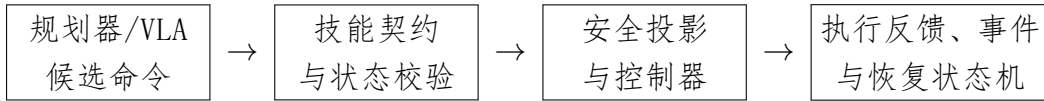


图 21.1：运行时间闭环。安全监督和恢复从状态与执行反馈独立取证，而不是只相信模型自报成功。

21.4 Simplex 思路：复杂性能控制与简单安全控制分离

Simplex 架构用高性能但难以完全验证的控制器处理正常任务，同时保留可验证的基线控制器；决策模块在安全边界前切换^[16]。在学习机器人中，高性能控制器可以是 VLA、扩散策略或 MPC，基线控制器可以是减速保持、撤退到已知自由空间或受控放置。

切换条件应包含前瞻裕度，而非等到约束已经违反。例如以制动距离 $d_b(v)$ 和感知/计算延迟 τ 估计

$$d_{\text{required}} = d_b(v) + v\tau + d_{\text{margin}}. \quad (21.4)$$

当障碍净距小于 d_{required} 时即抢占。这里的 τ 必须包括相机曝光、网络推理、队列、控制下发和执行器响应，而不只是模型推理时间。

21.5 异常分类决定恢复，而不是统一“再试一次”

恢复前先分类：可瞬态重试（短暂遮挡）、需主动感知（对象身份不确定）、需重规划（通道被占）、需补偿（抓取后目标变更）、需人工接管（风险/权限不明）、需锁存停机（驱动或安全链故障）。盲目重试可能把轻微滑移变成掉落，把碰撞接触变成持续施力。

Listing 21.1: 运行时监督器教学伪代码：抢占、降级与恢复

```

1 def supervise(command, state, health):
2     event = validate(command, state, health)
3     if event.is_hard_fault:
4         hardware_stop()
5         log_latched(event)
6         return STOPPED
7     if event.is_stale_state or event.identity_uncertain:
8         preempt_current_skill()
9         return request_active_perception(event)
10
11     safe_command, feasible = safety_filter(command, state)
12     if not feasible:
13         switch_to_verified_baseline("controlled_stop")
14         return RECOVERING
15
16     result = execute_with_deadline(safe_command)
17     log_event(command, safe_command, state.version, result)
18     if result.success and postcondition_observed(result.skill_id):
19         return SUCCEEDED
20     if result.code in RETRYABLE and retry_budget(result.skill_id) > 0:
21         return retry_with_updated_state(result)
22     if result.code in REPLANNABLE:
23         return request_replan(result.evidence)
24     switch_to_verified_baseline("safe_retreat")
25     return request_human(result)

```

事件日志至少记录：原始候选命令、过滤后命令、状态包版本、约束裕度、抢占者、失败码、传感证据和恢复结果。仅记录“任务失败”无法支持第 26 章的数据回采，也无法区分模型错误、接口错误和硬件错误。

21.6 系统案例：抓取策略只是运行时的一环

假设 VLA 输出未来 16 步末端动作。技能层先检查目标对象 ID 和工作区；轨迹层把动作块转换为时间参数化参考；安全层按最新障碍与力限值投影；控制器执行第一小段；视觉和触觉确认接近、接触与滑移；若对象身份变化则抢占，若轻微滑移则降低速度并调整夹持，若力传感器饱和则受控撤退。模型的贡献是提供候选行为，系统成功来自每一层都能拒绝、解释和恢复。

21.7 知识小结与综述判断

具身运行时把任务结构变成受约束的物理执行。技能契约明确边界，多时间尺度调度分配权限，安全过滤修改或拒绝建议动作，Simplex式基线控制提供降级路径，异常分类和事件日志支撑恢复与回采。综述判断是：任何“端到端”模型只要进入真实机器人，就必须回答谁能抢占、状态过期怎么办、约束无解怎么办、失败后如何确认环境；答不出这些问题，就还不是完整系统。

References

- [1] HE K, GKIOXARI G, DOLLAR P, et al. Mask R-CNN[C/OL]//2017 IEEE International Conference on Computer Vision. 2017: 2980–2988. DOI: [10.1109/ICCV.2017.322](https://doi.org/10.1109/ICCV.2017.322).
- [2] QI C R, SU H, MO K, et al. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation[C/OL]//2017 IEEE Conference on Computer Vision and Pattern Recognition. 2017: 652–660. DOI: [10.1109/CVPR.2017.16](https://doi.org/10.1109/CVPR.2017.16).
- [3] YUAN W, DONG S, ADELSON E H. GelSight: High-Resolution Robot Tactile Sensors for Estimating Geometry and Force[J/OL]. Sensors, 2017, 17(12): 2762. DOI: [10.3390/s17122762](https://doi.org/10.3390/s17122762).
- [4] KALMAN R E. A New Approach to Linear Filtering and Prediction Problems[J/OL]. Journal of Basic Engineering, 1960, 82(1): 35–45. DOI: [10.1115/1.3662552](https://doi.org/10.1115/1.3662552).
- [5] BARFOOT T D. State Estimation for Robotics[M/OL]. 2nd ed. Cambridge University Press, 2024. DOI: [10.1017/9781009299909](https://doi.org/10.1017/9781009299909).
- [6] HAFNER D, LILLICRAP T, FISCHER I, et al. Learning Latent Dynamics for Planning from Pixels[C/OL]//Proceedings of the 36th International Conference on Machine Learning: vol. 97. 2019: 2555–2565 [2026-07-14]. <https://proceedings.mlr.press/v97/hafner19a.html>.
- [7] HAFNER D, PASUKONIS J, BA J, et al. Mastering Diverse Domains through World Models[J/OL]. arXiv preprint arXiv:2301.04104, 2023. DOI: [10.48550/arXiv.2301.04104](https://doi.org/10.48550/arXiv.2301.04104).
- [8] ASSRAN M, DUVAL Q, MISRA I, et al. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture[C/OL]//2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2023: 15619–15629. DOI: [10.1109/CVPR52729.2023.01501](https://doi.org/10.1109/CVPR52729.2023.01501).
- [9] GHALLAB M, NAU D, TRAVERSO P. Automated Planning: Theory and Practice[M]. Morgan Kaufmann, 2004.

- [10] COLLEDANCHISE M, OGREN P. Behavior Trees in Robotics and AI: An Introduction[M/OL]. CRC Press, 2018. DOI: [10.1201/9780429489105](https://doi.org/10.1201/9780429489105).
- [11] AHN M, BROHAN A, BROWN N, et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances[J/OL]. Conference on Robot Learning, 2023, 205: 287–318 [2026-07-14]. <https://proceedings.mlr.press/v205/ahn23a.html>.
- [12] LIANG J, HUANG W, XIA F, et al. Code as Policies: Language Model Programs for Embodied Control[C/OL]//2023 IEEE International Conference on Robotics and Automation. 2023: 9493–9500. DOI: [10.1109/ICRA48891.2023.10160591](https://doi.org/10.1109/ICRA48891.2023.10160591).
- [13] HOGAN N. Impedance Control: An Approach to Manipulation. Part I—Theory[J/OL]. Journal of Dynamic Systems, Measurement, and Control, 1985, 107(1): 1–7. DOI: [10.1115/1.3140702](https://doi.org/10.1115/1.3140702).
- [14] KHATIB O. A Unified Approach for Motion and Force Control of Robot Manipulators: The Operational Space Formulation[J/OL]. IEEE Journal on Robotics and Automation, 1987, 3(1): 43–53. DOI: [10.1109/JRA.1987.1087068](https://doi.org/10.1109/JRA.1987.1087068).
- [15] AMES A D, XU X, GRIZZLE J W, et al. Control Barrier Function Based Quadratic Programs for Safety Critical Systems[J/OL]. IEEE Transactions on Automatic Control, 2017, 62(8): 3861–3876. DOI: [10.1109/TAC.2016.2638961](https://doi.org/10.1109/TAC.2016.2638961).
- [16] SHA L, GOPALAKRISHNAN S, LIU X, et al. Using Simplicity to Control Complexity[J/OL]. IEEE Software, 2001, 18(4): 20–28. DOI: [10.1109/52.936232](https://doi.org/10.1109/52.936232).